

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	Naveen Kumar S	Reg. No.:	192421030
Section / Batch:	B Tech IT	Date:	27.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I Naveen Kumar S - 192421030 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Naveen Kumar S

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

```

1 def exact_cover(universe, subsets, solution=None):
2     """
3     universe: set of elements that must be covered exactly once
4     subsets: list of sets (the collection of subsets to choose from)
5     solution: list to store the chosen subsets (used during recursion)
6     """
7     if solution is None:
8         solution = []
9
10    # Base case: universe is empty -> valid exact cover found
11    if not universe:
12        return solution
13
14    # Pruning: pick the element that appears in the fewest subsets
15    # This reduces branching factor significantly
16    element = min(universe, key=lambda e: sum(1 for s in subsets if e in s))
17
18    # Try every subset that contains this element
19    for subset in subsets:
20        if element in subset:
21            # Constraint check: subset must not overlap with already chosen subsets
22            if subset.issubset(universe):
23                # Choose this subset
24                new_universe = universe - subset
25                new_subsets = [s for s in subsets if not (s & subset)]
26

```

stdin input

Output

```

Subsets:
S1 = {1, 3, 5}
S2 = {2, 4}
S3 = {6, 7}
S4 = {1, 2, 3}
S5 = {4, 5, 6, 7}

```

EXPLANATION :

The **Exact Cover Problem** asks whether we can select some subsets from a collection such that **every element is covered exactly once**—not zero times and not more than once.

- **Coverage constraint:** Every element in the given universe must belong to at least one selected subset.
- **Exclusivity constraint:** No element can appear in two or more selected subsets.
- **Backtracking:** The algorithm tries different subsets one by one. If a choice violates the constraints, it goes back and tries another subset.
- **Pruning:** Unnecessary choices are removed early. For example, if a subset contains an element that has already been covered, that subset can be rejected immediately.
- **Correctness:** By exploring all valid possibilities while rejecting only choices that violate the constraints, backtracking can determine whether an exact cover exists.
- **Complexity:** Exact Cover is an **NP-Complete problem**, so its worst-case time complexity is generally **exponential**. Backtracking and pruning can significantly reduce the practical search space.

Example:

Suppose the elements are $\{1, 2, 3, 4\}$ and the subsets are $\{1,2\}$, $\{3,4\}$, $\{1,3\}$, and $\{2,4\}$. Selecting $\{1,2\}$ and $\{3,4\}$ covers every element **exactly once**, so an exact cover exists.

In short: The goal is to find a combination of subsets where **nothing is missed and nothing is covered twice**, using backtracking to efficiently search through possible combinations.